

SOFTWARE REQUIREMENTS SPECIFICATION

# Madrasa Management System

A multilingual, offline-first platform for student, teacher, and institution management

*Software Requirements Specification*

|                       |                                                                   |
|-----------------------|-------------------------------------------------------------------|
| System (working name) | Madrasa Management System (MMS) — working name, pending branding  |
| Document version      | v1.0                                                              |
| Status                | Draft for review                                                  |
| Date                  | 26 June 2026                                                      |
| Primary audience      | Engineering (FastAPI / React), product owner, future contributors |
| Backend / Frontend    | FastAPI (Python) backend · React PWA + Next.js marketing site     |
| Authored with         | Claude (Anthropic), co-authored with the product owner            |

*This specification describes the v1 build. Items explicitly marked Phase 2 or Out of Scope are recorded so the architecture leaves room for them without rework.*

## Table of Contents

---

# 1. Introduction

---

## 1.1 Purpose

This document specifies the requirements for the **Madrasa Management System (MMS)** — a web platform that lets a madrasa run its day-to-day academic and administrative operations: teacher and student records, attendance, class and course assignment, assignments, results, learning resources, forms, announcements, and voluntary finance. It is written for the engineers building the system and for anyone who later extends it, and it is detailed enough that a reader with no prior context can understand both *\*what\** is being built and *\*why\** the non-obvious decisions were made.

## 1.2 Document Conventions

- Requirements are grouped by module and given stable identifiers (for example **FR-ATT-05**). Identifiers are not renumbered when the list changes, so they can be referenced from issues and commits.
- Priority follows MoSCoW: **Must** (required for v1), **Should** (high value, include if feasible), **Could** (desirable, may slip).
- **Monospace** denotes code, identifiers, file names, configuration keys, and API paths.
- **v1**, **Phase 2**, and **Out of Scope** mark the delivery boundary. Phase 2 items are noted only where they affect a v1 design choice.

## 1.3 Scope

### *In scope for v1*

- A single madrasa, on a data model that is **multi-tenant-ready** so additional madaris can be onboarded later without restructuring.
- An installable **Progressive Web App (PWA)** for the Principal, teachers, and portal-enabled students, with **offline-first attendance** and a 7-day local data store.
- Three login roles — **Principal/Director/Head**, **Teacher** (capabilities granted individually by the Principal), and **Student** (portal access only where enabled) — plus **Guardians** as non-login contacts who receive WhatsApp messages.
- A separate, public **marketing website** (landing page, blog, contact, public registration) that is a normal server-rendered site, **not** a PWA.
- A configurable academic structure (programs, classes, sections, courses) and grading schemes with **no hardcoded subjects or numbers**.
- WhatsApp messaging via click-to-chat links ([wa.me](https://wa.me)) for sending performance reports and login credentials, with the messaging layer abstracted so the WhatsApp Business Cloud API can be added later.

### *Phase 2 (designed for, not built in v1)*

- Server-driven WhatsApp automation and file attachments via the WhatsApp Business Cloud API.
- A read-only guardian portal; automated payroll with absence-linked deductions; automatic timetable generation; activation of multi-tenant onboarding; per-period attendance; richer analytics.

### *Out of scope*

- Hostel / boarding management, online payment gateways, and a full general-purpose CMS are not part of this product at this time.

## 1.4 Intended Audience and Reading Order

Engineers should read Sections 3 (Architecture), 4 (Functional Requirements), and 6 (Data Model) in full. A product owner or reviewer can read Sections 1, 2, 5, and 8 for scope and quality expectations. Newcomers should start with the Glossary (Appendix B), which defines the madrasa-specific and technical terms used throughout.

## 1.5 References

- This specification is self-contained. External standards referenced by name only: the WhatsApp Business Platform (Cloud API) for Phase 2 messaging, the S3 object-storage API (implemented here with MinIO), and the Web App Manifest / Service Worker specifications underpinning the PWA.

## 2. Overall Description

### 2.1 Product Perspective

MMS is a new, self-contained product built from two cooperating surfaces that share one backend and one database:

- **The application (PWA).** The operational system used by staff and students. It is installable, works offline for attendance and key reads, and adapts its layout for Urdu (right-to-left) and English (left-to-right).
- **The marketing website.** A public, SEO-friendly site for the madrasa's presence — landing page, blog, contact, and a public admission registration form. It is server-rendered for search visibility and is deliberately *not* a PWA, because none of its visitors need offline use.

Both surfaces call the same FastAPI backend over a versioned REST API. Files (resources, assignment uploads, generated PDFs) live in S3-compatible object storage (MinIO). The whole system is containerised and deployed through Coolify.

### 2.2 Product Functions (summary)

| Area              | What it does                                                                                                                       |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Identity & access | Principal-created logins; granular per-teacher permissions; portal access toggled per student; guardians as message-only contacts. |
| Academic setup    | Admin-defined programs, classes, sections, courses, and named academic sessions (Hijri + Gregorian).                               |
| Teacher records   | Profiles, attendance with check-in/out timings and a daily lock, and salary kept in its own module.                                |
| Student records   | Profiles, enrolment, courses, timetable, attendance, and results.                                                                  |
| Teaching workflow | Assign teachers to class+course; create and grade assignments; build and open forms for classes.                                   |
| Assessment        | Dynamic exam types per course with weightage; configurable grading schemes; published results and printable result cards.          |
| Content           | Categorised resources (notes, PDFs, video links); internal announcements; a public blog.                                           |
| Finance           | A voluntary income ledger for student contributions and community donations (free by default).                                     |
| Messaging         | One-tap WhatsApp performance reports and credential delivery via click-to-chat links.                                              |

### 2.3 User Classes and Characteristics

| User class                  | Login | Characteristics & needs                                                                                                                                         |
|-----------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Principal / Director / Head | Yes   | The superuser. Configures the madrasa, creates all logins, grants teacher permissions, and oversees everything. May have limited time and delegate heavily.     |
| Teacher                     | Yes   | Base role plus exactly the capabilities the Principal grants (checkboxes). Most actions are limited to the classes/courses they are assigned; some teachers are |

| User class        | Login                     | Characteristics & needs                                                                                                                                                      |
|-------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                   |                           | given supervisory or institution-wide capabilities.                                                                                                                          |
| Student           | Portal only where enabled | Sees their own timetable, results, assignments, resources, forms, and announcements. Younger students may have no portal at all — portal access is configurable per student. |
| Guardian / Parent | No (contact only)         | Receives WhatsApp messages about their child's performance and credentials. Identified by phone number and preferred language. May vary widely in literacy and device type.  |

## 2.4 Operating Environment

- **Clients:** modern mobile and desktop browsers (Chrome, Edge, Safari, Firefox). The PWA targets Android devices in particular, since teachers will mark attendance on phones, sometimes offline.
- **Server:** Linux containers orchestrated by Coolify; FastAPI behind an ASGI server, PostgreSQL, Redis, and MinIO.
- **Connectivity:** intermittent. The app must remain useful for a full working day with no connectivity and reconcile when it returns.

## 2.5 Design and Implementation Constraints

- **Bilingual, with true RTL.** Urdu is right-to-left; supporting it means mirroring layout, not just translating strings. This is a first-class constraint, not a later polish (see 3.6).
- **Dynamic and modular.** No subject, class size, exam count, or grade band may be hardcoded. New features must be addable as self-contained modules without touching unrelated code (see 5.6).
- **Offline-first.** Attendance capture cannot depend on the network (see 3.4).
- **Free messaging in v1.** WhatsApp delivery uses click-to-chat links and the teacher's own WhatsApp — no paid API, no message templates approval, no per-message cost — at the price of requiring a human tap to send.
- **Stated stack.** FastAPI (Python) backend and React frontend are fixed by the team's expertise; the marketing site uses Next.js for server rendering.

## 2.6 Assumptions and Dependencies

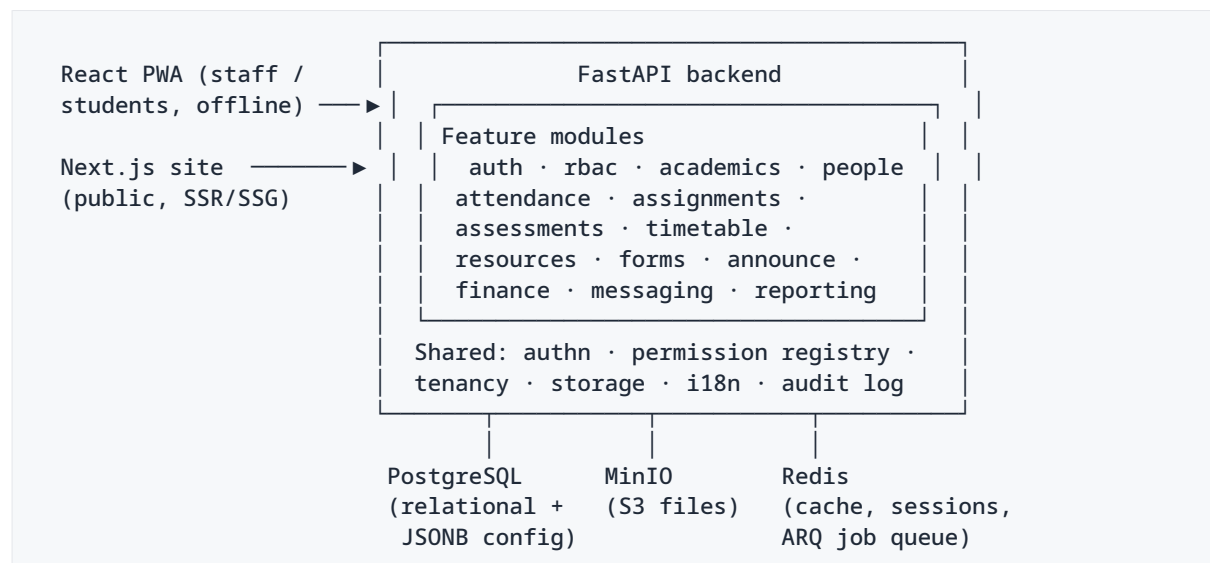
- Staff phones can run a modern browser and have WhatsApp installed for the click-to-chat flow.
- Guardian and student phone numbers are collected accurately, since WhatsApp delivery depends on them.
- The madrasa is comfortable with self-hosting (Coolify) and providing storage for uploaded files and backups.
- Exact grade bands, programs, and class structure are configured by the Director at setup time rather than assumed by the system.

## 3. System Architecture

### 3.1 Architectural Overview

MMS is a **modular monolith**: one FastAPI application internally divided into independent feature modules, paired with a React PWA and a Next.js marketing site. A modular monolith — rather than microservices — fits a small madrasa and a one-to-two-person team: simple to deploy and reason about, while clean module boundaries keep the door open to extraction later.

Each backend module owns its routes, service logic, database models, and schemas, and registers its permissions and any background jobs through shared registries. Cross-cutting concerns — authentication, the permission registry, tenancy scoping, file storage, messaging, internationalisation, and the audit log — are provided as shared services that every module consumes.



### 3.2 Technology Stack

| Layer             | Choice                   | Why                                                                                                                                                                                            |
|-------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend framework | FastAPI (async)          | Team expertise; async suits offline-sync bursts; first-class OpenAPI for the React client.                                                                                                     |
| ORM / migrations  | SQLAlchemy 2.x + Alembic | Mature, async-capable, versioned schema changes.                                                                                                                                               |
| Validation        | Pydantic v2              | Request/response schemas and config validation.                                                                                                                                                |
| Database          | PostgreSQL 16            | Relational integrity for core entities plus JSONB for the dynamic parts (grading schemes, form definitions, exam-type config). MySQL 8 is viable but JSONB makes Postgres the better fit here. |
| Cache / queue     | Redis + ARQ              | Sessions, caching, and async jobs (PDF generation, future messaging) in the asyncio model.                                                                                                     |
| Object storage    | MinIO (S3 API)           | Self-hosted, S3-compatible; a local-disk driver covers development and the offline fallback (3.5).                                                                                             |
| App frontend      | React + Vite (PWA)       | Stated preference; Vite + Workbox for the service worker; TanStack Query for server state and offline                                                                                          |

| Layer          | Choice                   | Why                                                                                      |
|----------------|--------------------------|------------------------------------------------------------------------------------------|
|                |                          | caching; Dexie over IndexedDB for the local store.                                       |
| Marketing site | Next.js                  | Server rendering / static generation for blog SEO that a plain React SPA cannot provide. |
| i18n           | i18next / react-i18next  | Runtime language switching, RTL handling, pluralisation.                                 |
| Deployment     | Coolify + Docker Compose | Self-hosted PaaS; one compose stack per environment (3.9).                               |

### 3.3 Multi-Tenant-Ready Design

v1 serves one madrasa, but every tenant-scoped table carries a `madrasa_id` foreign key and all data access is filtered through it. In v1 there is exactly one madrasa row; activating multi-tenancy later means onboarding additional rows and binding requests to a tenant — no schema rewrite. A shared-schema, shared-database model is chosen over schema-per-tenant because it keeps migrations and operations simple at this scale.

**Note** `madrasa_id` is applied as a default query filter at the data-access layer so individual queries cannot forget it. Global/reference data (e.g., system permission definitions) is the only category that is not tenant-scoped.

### 3.4 Offline-First and Synchronization

Teachers must be able to take attendance with no connectivity and have it reach the server later. This is the most demanding part of the system, so its rules are precise.

#### *Local capability*

- The PWA caches each teacher's assigned class rosters, timetable, and recent reference data, and keeps a **7-day local store** (Dexie/IndexedDB) so the app is usable offline for a **full working day** and beyond.
- Offline writes (attendance, and queued reads/edits) are recorded locally with an on-device timestamp and an idempotency key, then placed in an outbox.
- When connectivity returns, the outbox syncs in the background; each item is acknowledged and cleared. Idempotency keys make re-sends safe.

#### *The daily attendance lock and how it survives offline*

A day's attendance **locks at 23:59 of that day**, after which entries are read-only. The lock keys off the **local capture time**, not the sync time — otherwise a teacher who is legitimately offline for several days could never submit. The rules:

1. **Online:** you cannot create or alter attendance for a past, already-locked date from scratch. Corrections to a still-open day are allowed and logged.
2. **Offline-captured:** the record carries the device's capture timestamp. On sync it is accepted even though the date has passed, and is flagged `synced_late` with the sync timestamp recorded in the audit log.
3. **Post-lock override:** only the Principal can edit locked attendance, and only with a mandatory reason, fully audited.

This closes the obvious fraud gap (back-dating a present mark days later online) while honouring the offline-first requirement.



### Missing-data visibility

- The Principal/Director sees a **missing-attendance indicator** that lists any teacher whose attendance has not synced for \*N\* days, so an offline or non-compliant device is visible rather than silently absent.

### Conflict handling

- Attendance is effectively conflict-free (one teacher per class roster per day). **Marks** can collide across a multi-day offline gap; the rule is **last-write-wins plus a full audit entry**, surfacing the overwritten value ("marks previously entered by X: old → new by Y") and flagging when the server value changed after the offline copy was taken. Nothing is silently lost.

## 3.5 File Storage Abstraction

All binary content (resource files, assignment uploads, generated result-card PDFs) is written through a **StorageProvider** interface with two drivers: an **S3 driver (MinIO)** as the primary store, and a **local-filesystem driver** for development and as a fallback. Files are referenced by object keys in the database; downloads to guardians/students use **time-limited signed URLs** rather than public links. On the client, the PWA may cache needed files locally during the offline window so resources remain viewable without connectivity.

## 3.6 Internationalization and RTL

- **UI chrome** (menus, labels, buttons, system messages) is fully bilingual — Urdu and English — switchable at runtime per user.
- **Direction:** Urdu renders the entire interface right-to-left (mirrored navigation, tables, forms, icons); English renders left-to-right. The `dir` attribute and logical CSS properties drive this, not per-component hacks.
- **Fonts:** an Urdu Nasta'liq face (e.g., Noto Nastaliq Urdu / Jameel Noori Nastaleeq) for Urdu text; a clean Latin face for English.
- **Formatting:** dates show **Hijri alongside Gregorian**; numerals, and where appropriate Eastern-Arabic digits, follow the active language.
- **Content language** (course names, blog posts, resource titles, form questions) uses a single Director-selected language: if the Director sets Urdu as the content language, all entered content is captured in Urdu. The UI remains per-user bilingual regardless.
- **WhatsApp templates** are maintained in both languages with variable placeholders and are sent in the guardian's or student's preferred language.

## 3.7 Messaging Layer

Messaging is an abstraction with pluggable drivers. v1 ships the **wa.me click-to-chat driver**: the app builds a link `https://wa.me/<number>?text=<encoded message>` from a bilingual template, and the staff member taps Send in their own WhatsApp. It is free, needs no API approval, and works for performance reports and credential delivery, but it sends **text only** (PDFs are linked, not attached) and requires a human tap. Phase 2 adds a **Cloud API driver** for server-sent, automated, attachment-capable messages using Meta-approved templates — no change to calling code, since both implement the same interface.

**Note** Phone numbers are normalised to international digits-only form before building links (Pakistan: 92 + number, leading zero dropped → 923001234567).

### 3.8 Security, Privacy, and Audit

- **Credentials are never sent in plaintext.** The "send credentials" action delivers a username plus a **one-time, time-limited set-password link (1-day expiry)**; the user sets their own password on first login. The same 1-day expiry applies to any tokenised link.
- **Authorization is server-side on every endpoint** via the permission registry; the UI only hides actions a user cannot perform — it never substitutes for server checks.
- **Audit log.** Attendance entries and corrections, mark changes, finance entries, permission grants, and login provisioning are recorded immutably (who, what, when, old → new). This is the backbone of accountability and dispute resolution.
- **Minors' data.** Student records concern children; access is least-privilege, files are served via signed URLs, and personal data is not exposed on public surfaces.
- **Transport & sessions:** HTTPS throughout (terminated at Coolify's proxy), hashed passwords (argon2/bcrypt), short-lived sessions/tokens, rate limiting and lockout on auth endpoints.

### 3.9 Deployment Architecture (Coolify)

The stack is deployed on **Coolify** from a Docker Compose definition. Coolify provides the reverse proxy, TLS certificates, environment-variable management, and per-service deploys. The compose stack below is the canonical reference; the same files are provided as standalone artifacts alongside this document.

#### 3.9.1 *docker-compose.yml*

```
services:
  backend:
    build: { context: ./backend, dockerfile: Dockerfile }
    environment:
      - DATABASE_URL=postgresql+asyncpg://mms:${DB_PASSWORD}@db:5432/mms
      - REDIS_URL=redis://redis:6379/0
      - S3_ENDPOINT=http://minio:9000
      - S3_BUCKET=mms-files
      - S3_ACCESS_KEY=${MINIO_ROOT_USER}
      - S3_SECRET_KEY=${MINIO_ROOT_PASSWORD}
      - SECRET_KEY=${SECRET_KEY}
      - DEFAULT_TENANT=default
    depends_on: [db, redis, minio]
    restart: unless-stopped

  worker:
    # ARQ background jobs (PDFs, future messaging)
    build: { context: ./backend, dockerfile: Dockerfile }
    command: arq app.worker.WorkerSettings
    environment:
      - DATABASE_URL=postgresql+asyncpg://mms:${DB_PASSWORD}@db:5432/mms
      - REDIS_URL=redis://redis:6379/0
    depends_on: [db, redis]
    restart: unless-stopped

  app:
    # React PWA (built, served by nginx)
    build: { context: ./app, dockerfile: Dockerfile }
    restart: unless-stopped

  web:
    # Next.js marketing site (SSR)
    build: { context: ./web, dockerfile: Dockerfile }
    environment:
```

```

    - NEXT_PUBLIC_API_BASE=${API_BASE}
restart: unless-stopped

db:
  image: postgres:16-alpine
  environment:
    - POSTGRES_USER=mms
    - POSTGRES_PASSWORD=${DB_PASSWORD}
    - POSTGRES_DB=mms
  volumes: [ db-data:/var/lib/postgresql/data ]
  restart: unless-stopped

redis:
  image: redis:7-alpine
  restart: unless-stopped

minio:
  image: minio/minio
  command: server /data --console-address ":9001"
  environment:
    - MINIO_ROOT_USER=${MINIO_ROOT_USER}
    - MINIO_ROOT_PASSWORD=${MINIO_ROOT_PASSWORD}
  volumes: [ minio-data:/data ]
  restart: unless-stopped

volumes:
  db-data:
  minio-data:

```

### 3.9.2 backend/Dockerfile

```

FROM python:3.12-slim
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
  build-essential libpq-dev && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "app.main:app", "-k", "uvicorn.workers.UvicornWorker", \
  "-w", "4", "-b", "0.0.0.0:8000"]

```

### 3.9.3 app/Dockerfile (React PWA → static, nginx)

```

FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf # SPA fallback to index.html
EXPOSE 80

```

### 3.9.4 web/Dockerfile (Next.js marketing site)

```
FROM node:20-alpine AS build
WORKDIR /web
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /web
COPY --from=build /web/.next ./.next
COPY --from=build /web/node_modules ./node_modules
COPY --from=build /web/package.json ./package.json
COPY --from=build /web/public ./public
EXPOSE 3000
CMD ["npm", "start"]
```

### 3.9.5 Environment variables (.env)

| Variable                                 | Purpose                                                  |
|------------------------------------------|----------------------------------------------------------|
| DB_PASSWORD                              | PostgreSQL password for the mms user.                    |
| SECRET_KEY                               | Backend signing key for tokens/sessions.                 |
| MINIO_ROOT_USER /<br>MINIO_ROOT_PASSWORD | MinIO credentials, also passed to the backend S3 driver. |
| API_BASE /<br>NEXT_PUBLIC_API_BASE       | Public base URL of the backend API for the frontends.    |
| DEFAULT_TENANT                           | Slug of the single v1 madrasa row.                       |

### 3.9.6 Coolify notes

- Each service maps to a Coolify resource; the proxy routes the app and marketing domains and issues TLS certificates automatically.
- `db-data` and `minio-data` are persistent volumes; include both in the backup routine (5.10), along with periodic `pg_dump`.
- Set environment variables in Coolify's secrets rather than committing them; this file lists names only.
- Bucket `mms-files` is created on first boot (init job or console); the local-disk fallback driver is used only outside this stack (e.g., local dev).

## 4. Functional Requirements

Requirements are grouped by module. Each module lists its requirements with a stable ID and a MoSCoW priority. Where a rule has subtlety (attendance locking, marks sync, permission scope), the prose around the table is normative.

### 4.1 Authentication & User Provisioning

All logins are created by the **Admin** — the Principal, or a teacher granted the provisioning permission. There is no open self-service login creation inside the app (the public marketing site has a separate admission application, 4.18). When a student is added, a login record is generated and stored; whether that student can actually use the portal is governed by a per-student flag.

| ID         | Requirement                                                                                                                                                                                    | Pri    |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| FR-AUTH-01 | The Admin creates Principal, Teacher, and Student logins. No in-app self-registration of logins exists.                                                                                        | Must   |
| FR-AUTH-02 | On creating a student, the system generates and stores login details; a per-student <b>portal-enabled</b> flag controls whether the student may sign in (default settable at class level).     | Must   |
| FR-AUTH-03 | A <b>Send credentials</b> action dispatches the username plus a one-time, 1-day set-password link to the student's and/or guardian's number via WhatsApp (4.16). Raw passwords are never sent. | Must   |
| FR-AUTH-04 | On first sign-in the user must set their own password; the set-password link expires after 1 day and on use.                                                                                   | Must   |
| FR-AUTH-05 | Login is by username + password. The user's preferred interface language is stored and applied on sign-in.                                                                                     | Must   |
| FR-AUTH-06 | Sessions/tokens are short-lived and renewable; sign-out invalidates the session.                                                                                                               | Must   |
| FR-AUTH-07 | Auth endpoints enforce rate limiting and temporary lockout after repeated failures.                                                                                                            | Should |
| FR-AUTH-08 | Phone-OTP login is available as a configurable alternative for audiences without reliable usernames/passwords.                                                                                 | Could  |
| FR-AUTH-09 | Guardians have no login in v1; they exist as message-only contacts.                                                                                                                            | Must   |

### 4.2 Roles & Permissions

The Principal is an implicit superuser. Teachers receive capabilities individually through a **granular permission catalogue** presented as checkboxes. Capabilities split into two kinds: **scoped** capabilities apply only to the teacher's own class/course assignments (e.g., taking attendance, entering marks, creating assignments), while **supervisory/global** capabilities deliberately bypass that scope (e.g., managing all assignments, adding teachers, managing the blog). The full catalogue is in Appendix A.

| ID         | Requirement                                                                                                                      | Pri  |
|------------|----------------------------------------------------------------------------------------------------------------------------------|------|
| FR-RBAC-01 | The Principal holds all permissions implicitly and cannot be locked out of administration.                                       | Must |
| FR-RBAC-02 | The Principal grants/revokes individual permissions per teacher via checkboxes; changes take effect immediately and are audited. | Must |

| ID         | Requirement                                                                                                                                                                           | Pri  |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| FR-RBAC-03 | Scoped capabilities are enforced against the teacher's assignments; a teacher cannot act on a class/course they are not assigned to unless holding a matching supervisory permission. | Must |
| FR-RBAC-04 | Supervisory/global permissions (e.g., <code>assignments.manage_all</code> , <code>teachers.add</code> , <code>blog.manage</code> ) explicitly bypass assignment scope.                | Must |
| FR-RBAC-05 | Every API endpoint enforces its permission server-side; the UI hides actions the current user lacks, but server checks are authoritative.                                             | Must |
| FR-RBAC-06 | The permission catalogue is extensible: new modules register new permissions without altering existing grants.                                                                        | Must |
| FR-RBAC-07 | Granting <code>blog.manage</code> automatically reveals a <b>Manage Blog</b> entry in that teacher's menu (4.18).                                                                     | Must |

### 4.3 Academic Structure & Sessions

The academic structure is fully data-defined so the system suits any madrasa. The hierarchy is **Program** → **Class/Darja** → **Section** → **Course/Subject**, all created by an authorised user. Nothing about subjects, class counts, or sizes is hardcoded.

| ID         | Requirement                                                                                                             | Pri  |
|------------|-------------------------------------------------------------------------------------------------------------------------|------|
| FR-ACAD-01 | Define and edit <b>Programs</b> (e.g., Hifz, Nazira, Dars-e-Nizami) with no fixed list.                                 | Must |
| FR-ACAD-02 | Define <b>Classes/Darjat</b> within a program and <b>Sections</b> within a class.                                       | Must |
| FR-ACAD-03 | Define <b>Courses/Subjects</b> and attach them to classes; subjects are arbitrary and unlimited.                        | Must |
| FR-ACAD-04 | Define <b>Academic Sessions</b> with a human name and Hijri + Gregorian span; mark exactly one session active.          | Must |
| FR-ACAD-05 | Enrol students into a program/class/section for a session; an enrolment determines the student's courses and timetable. | Must |
| FR-ACAD-06 | All structure entities are tenant-scoped via <code>madrasa_id</code> (3.3).                                             | Must |

### 4.4 Teacher Management

#### 4.4.1 Records

| ID        | Requirement                                                                                                              | Pri  |
|-----------|--------------------------------------------------------------------------------------------------------------------------|------|
| FR-TCH-01 | Create/edit teacher profiles: name, contact (incl. WhatsApp number), qualifications, join date, status, and notes.       | Must |
| FR-TCH-02 | Assign each teacher an <b>employee code</b> , auto-generated in a Director-defined format, with manual override allowed. | Must |
| FR-TCH-03 | List, search, and filter teachers; deactivate without deleting (records and history are retained).                       | Must |

#### 4.4.2 Teacher Attendance

Teacher attendance records a daily status with arrival timing and is subject to the same end-of-day lock as student attendance, with corrections fully logged.

| ID            | Requirement                                                                                                                                                                                       | Pri    |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| FR-TCH-ATT-01 | Record daily teacher status: Present / Absent / Leave, per session date.                                                                                                                          | Must   |
| FR-TCH-ATT-02 | On Present, capture a <b>check-in time</b> and an optional <b>check-out time</b> .                                                                                                                | Must   |
| FR-TCH-ATT-03 | Store <b>marked_at</b> (capture timestamp) and <b>marked_by</b> for every entry.                                                                                                                  | Must   |
| FR-TCH-ATT-04 | Allow changing a status (e.g., Absent → Present) before the day lock; each change writes a <b>correction-log</b> entry with previous value, new value, actor, and the <b>time of the change</b> . | Must   |
| FR-TCH-ATT-05 | <b>Lock</b> the day's attendance at 23:59 of that date; thereafter entries are read-only.                                                                                                         | Must   |
| FR-TCH-ATT-06 | For offline-captured attendance, key the lock to the on-device capture time; accept late syncs of past dates and flag them <b>synced_late</b> with the sync timestamp (3.4).                      | Must   |
| FR-TCH-ATT-07 | Permit a <b>Principal-only</b> post-lock override requiring a reason; record it in the audit log.                                                                                                 | Should |
| FR-TCH-ATT-08 | Show the Director a <b>missing-attendance</b> view: teachers whose attendance has not synced for N days.                                                                                          | Must   |

#### 4.4.3 Salary (separate module)

Salary is kept **separate from the Finance income ledger** (4.15). It lives on the teacher record as a figure plus a payment history.

| ID        | Requirement                                                                              | Pri    |
|-----------|------------------------------------------------------------------------------------------|--------|
| FR-SAL-01 | Maintain a salary figure per teacher (amount, currency, effective period).               | Must   |
| FR-SAL-02 | Record salary payment entries: date, amount, period covered, method, note.               | Must   |
| FR-SAL-03 | Keep salary entirely independent of the Finance module's income/donation records.        | Must   |
| FR-SAL-04 | Teacher absences are visible to the Director but do <b>not</b> auto-deduct salary in v1. | Should |
| FR-SAL-05 | Absence-linked deduction rules and full payroll are Phase 2.                             | Could  |

### 4.5 Student Management

| ID        | Requirement                                                                                                 | Pri  |
|-----------|-------------------------------------------------------------------------------------------------------------|------|
| FR-STU-01 | Create/edit student profiles: name, date of birth, contact, guardian link(s), enrolment, status, and notes. | Must |
| FR-STU-   | Assign an <b>admission number</b> , auto-generated in a Director-defined                                    | Must |

| ID        | Requirement                                                                                                                                                                                                              | Pri  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| 02        | format, with manual override.                                                                                                                                                                                            |      |
| FR-STU-03 | Set <b>portal access</b> per student; default the setting at class level (e.g., a young class defaults to no portal), overridable per student.                                                                           | Must |
| FR-STU-04 | Record daily student attendance: Present / Absent / Leave, <b>per day</b> (not per period), with <b>marked_at</b> , <b>marked_by</b> , a correction log, and the 23:59 lock with the same offline-capture rule as 4.4.2. | Must |
| FR-STU-05 | A student's enrolment yields their course list and timetable automatically; results attach to the active session.                                                                                                        | Must |
| FR-STU-06 | Portal-enabled students see their own profile, timetable, attendance, results, assignments, resources, forms, and announcements — and only their own.                                                                    | Must |
| FR-STU-07 | List, search, filter, and deactivate students without deletion.                                                                                                                                                          | Must |

## 4.6 Guardian Management

| ID        | Requirement                                                                                                                             | Pri   |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------|-------|
| FR-GRD-01 | Create/edit guardian contacts: name, relationship, one or more phone numbers, and <b>preferred language</b> .                           | Must  |
| FR-GRD-02 | Link a guardian to one or more students (and a student to one or more guardians).                                                       | Must  |
| FR-GRD-03 | Guardians receive WhatsApp performance messages and credential dispatches in their preferred language (4.16); they have no login in v1. | Must  |
| FR-GRD-04 | A read-only guardian portal is Phase 2; the data model must allow promoting a guardian to a login later.                                | Could |

## 4.7 Class & Course Assignment

| ID        | Requirement                                                                                    | Pri  |
|-----------|------------------------------------------------------------------------------------------------|------|
| FR-ASN-01 | Assign a teacher to a specific <b>class + course</b> combination for the active session.       | Must |
| FR-ASN-02 | A teacher may hold many assignments; a class+course may have a designated teacher.             | Must |
| FR-ASN-03 | Scoped capabilities (attendance, marks, assignments) are bounded by these assignments (4.2).   | Must |
| FR-ASN-04 | Reassignment preserves historical records (past attendance/marks remain attributed correctly). | Must |

## 4.8 Attendance (cross-cutting behaviour)

Attendance for both students and teachers shares one engine; module-specific fields differ (teacher timings). The cross-cutting rules below are normative and complement 4.4.2 and FR-STU-04.



| ID        | Requirement                                                                                                                                | Pri  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------|------|
| FR-ATT-01 | Capture attendance <b>offline</b> ; persist locally with an on-device timestamp and idempotency key; sync automatically when online (3.4). | Must |
| FR-ATT-02 | The local store retains data for <b>7 days</b> and supports a full offline working day.                                                    | Must |
| FR-ATT-03 | Apply the <b>23:59 lock</b> by local capture date; accept and flag <code>synced_late</code> past-dated offline records.                    | Must |
| FR-ATT-04 | Every create/correction is written to the audit log with actor, timestamps, and old → new values.                                          | Must |
| FR-ATT-05 | Provide attendance summaries per student, class, and date range, excluding holidays/approved leave (4.11.x leave & holidays).              | Must |

## 4.9 Assignments & Submissions

| ID        | Requirement                                                                                                                                                         | Pri    |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| FR-ASG-01 | A teacher creates an assignment for an assigned class+course: title, instructions, optional attachment, due date.                                                   | Must   |
| FR-ASG-02 | Target the <b>whole class</b> or <b>specific students</b> within it.                                                                                                | Must   |
| FR-ASG-03 | Portal-enabled students view their assignments and <b>upload submissions</b> (files → object storage), with resubmission allowed until the due date (configurable). | Must   |
| FR-ASG-04 | Teachers view submissions, record a mark and/or feedback; an assignment may optionally feed a gradebook entry (4.10).                                               | Must   |
| FR-ASG-05 | Teachers with a supervisory permission can <b>view, edit, and create assignments for any class</b> — head teachers oversee or set assignments themselves.           | Must   |
| FR-ASG-06 | Late submissions are flagged relative to the due date.                                                                                                              | Should |

## 4.10 Assessments & Results

Assessment is dynamic end to end: the Director defines the grading scheme(s); teachers define however many exam types a course needs; weighted assessments roll up into a course result, and course results aggregate into a session result. Raw scores map to grade bands through the active scheme.

### 4.10.1 Grading schemes

| ID        | Requirement                                                                                                                                                            | Pri  |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| FR-RES-01 | The Director defines one or more <b>grading schemes</b> : percentage, letter grades, or madrasa-style bands (e.g., Mumtaz / Jayyid Jiddan / Jayyid / Maqbool / Rasib). | Must |
| FR-RES-02 | A scheme defines ordered bands with score thresholds and labels; schemes are stored as configurable data (JSONB), not code.                                            | Must |
| FR-RES-03 | A scheme is selectable per program and/or per exam, so different parts of the madrasa can grade differently.                                                           | Must |

### 4.10.2 Exam types, marks, and results

| ID        | Requirement                                                                                                                                                  | Pri  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| FR-RES-04 | A teacher creates <b>1..N exam types</b> per course (e.g., one, or ten), each with a name and an optional <b>weightage</b> .                                 | Must |
| FR-RES-05 | Enter marks per student per assessment; entry is scoped to the assigned teacher.                                                                             | Must |
| FR-RES-06 | Roll up weighted assessments into a course result; aggregate course results into a per-student <b>session result</b> .                                       | Must |
| FR-RES-07 | Map computed scores to grade bands using the active scheme; show both raw and banded results.                                                                | Must |
| FR-RES-08 | <b>Publish</b> results under permission control; portal students then see their results.                                                                     | Must |
| FR-RES-09 | Generate a downloadable <b>result-card PDF</b> (bilingual, Hijri + Gregorian) for sharing with off-portal guardians via a WhatsApp link.                     | Must |
| FR-RES-10 | On offline marks sync, apply <b>last-write-wins + audit</b> , surfacing the overwritten value and flagging server-side changes since the offline copy (3.4). | Must |

### 4.11 Timetable, Leave & Holiday Calendar

Timetables are created manually in v1 (the madrasa is small and manual entry is sufficient); automatic generation is Phase 2. A holiday calendar and leave handling keep attendance statistics honest.

| ID       | Requirement                                                                                                            | Pri   |
|----------|------------------------------------------------------------------------------------------------------------------------|-------|
| FR-TT-01 | An authorised user builds a timetable manually for a class/section: day, period/time, course, teacher.                 | Must  |
| FR-TT-02 | Students and teachers view their own timetable; the PWA caches it for offline viewing.                                 | Must  |
| FR-TT-03 | Maintain a <b>holiday calendar</b> ; days marked as holidays are excluded from attendance expectations and statistics. | Must  |
| FR-TT-04 | Record <b>leave</b> for students and teachers; approved leave is distinguished from absence in summaries.              | Must  |
| FR-TT-05 | Automatic timetable generation from constraints is Phase 2.                                                            | Could |

### 4.12 Resources

| ID        | Requirement                                                                                                           | Pri  |
|-----------|-----------------------------------------------------------------------------------------------------------------------|------|
| FR-RSC-01 | Maintain an <b>open resource library</b> of notes, PDFs/files, and video links, organised by configurable categories. | Must |
| FR-RSC-02 | Files are stored in object storage; video links are stored as URLs.                                                   | Must |
| FR-RSC-03 | Scope a resource's visibility to specific classes/courses or make it generally available.                             | Must |
| FR-RSC-04 | Portal students browse and open resources visible to them; cached for offline where applicable.                       | Must |

### 4.13 Form Builder

A **simple** form builder — not a logic engine. It offers the minimal HTML controls a madrasa needs to collect structured input.

| ID        | Requirement                                                                                                                                                                | Pri   |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| FR-FRM-01 | Create a form from basic fields: <b>label, text field, text area, radio group, checkbox group, dropdown, and a submit button</b> ; each field can be required or optional. | Must  |
| FR-FRM-02 | <b>Open</b> a form for specific class(es)/students, with an optional open/close window.                                                                                    | Must  |
| FR-FRM-03 | Collect responses; configure whether a student may submit once or multiple times.                                                                                          | Must  |
| FR-FRM-04 | View and export responses under permission control.                                                                                                                        | Must  |
| FR-FRM-05 | Conditional logic / branching is Phase 2.                                                                                                                                  | Could |

### 4.14 Announcements

| ID        | Requirement                                                                                                 | Pri    |
|-----------|-------------------------------------------------------------------------------------------------------------|--------|
| FR-ANN-01 | An authorised user posts an announcement to <b>all</b> users or to a <b>specific class/section</b> .        | Must   |
| FR-ANN-02 | An announcement has a title, body, optional attachment link, and optional schedule/expiry.                  | Must   |
| FR-ANN-03 | Announcements appear in the relevant users' portal feed; optional WhatsApp/push delivery (push is Phase 2). | Should |

### 4.15 Finance (voluntary income ledger)

Finance is a **standalone ledger of voluntary income**, not a per-student fee obligation. Contribution is free by default; those who wish to pay may, and external community members may donate. Teacher salary is **not** part of this module (4.4.3).

| ID        | Requirement                                                                                                                                    | Pri    |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| FR-FIN-01 | Record a <b>student contribution</b> : linked student, amount, date, category/purpose, note; contribution is voluntary with no auto-invoicing. | Must   |
| FR-FIN-02 | Record a <b>community/external donation</b> : donor record (name, contact), amount, date, purpose — not linked to any student.                 | Must   |
| FR-FIN-03 | Maintain configurable <b>categories/purposes</b> for income.                                                                                   | Must   |
| FR-FIN-04 | Issue a <b>receipt</b> (printable / PDF) for a contribution or donation; shareable via a WhatsApp link.                                        | Should |
| FR-FIN-05 | Report totals and breakdowns: who contributed/donated how much, by period and category.                                                        | Must   |
| FR-FIN-06 | Salary disbursement is excluded from this ledger and handled in the salary module.                                                             | Must   |

## 4.16 Messaging (WhatsApp)

v1 uses **click-to-chat** links so messages cost nothing and use the staff member's own WhatsApp; the trade-off is that each send needs a human tap. The layer is abstracted for a Phase 2 Cloud API driver (3.7).

| ID        | Requirement                                                                                                                                                                                  | Pri    |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| FR-MSG-01 | From a student's performance context, <b>Send report to parents</b> opens WhatsApp to the guardian's number with a <b>prefilled, templated</b> message in the guardian's preferred language. | Must   |
| FR-MSG-02 | Build links as <code>https://wa.me/&lt;number&gt;?text=&lt;encoded&gt;</code> ; <b>normalise</b> numbers to international digits-only form first.                                            | Must   |
| FR-MSG-03 | Maintain <b>bilingual message templates</b> with variable placeholders (student name, marks, links).                                                                                         | Must   |
| FR-MSG-04 | <b>Send credentials</b> dispatches username + a 1-day one-time set-password link (never a raw password) via WhatsApp.                                                                        | Must   |
| FR-MSG-05 | Messages are <b>text only</b> in v1; PDFs (result cards, receipts) are shared as signed links, not attachments.                                                                              | Must   |
| FR-MSG-06 | Log each dispatch (template, recipient, sender, time). Delivery confirmation is not available with click-to-chat; the log records intent/open.                                               | Should |
| FR-MSG-07 | Server-sent automation and attachments (Cloud API) are Phase 2 behind the same interface.                                                                                                    | Could  |

## 4.17 Dashboards & Reporting

| ID        | Requirement                                                                                                 | Pri    |
|-----------|-------------------------------------------------------------------------------------------------------------|--------|
| FR-RPT-01 | <b>Principal dashboard:</b> key counts, the missing-attendance indicator, finance summary, recent activity. | Must   |
| FR-RPT-02 | <b>Teacher dashboard:</b> my classes, take-attendance shortcuts, pending submissions, my timetable.         | Must   |
| FR-RPT-03 | <b>Student dashboard:</b> today's timetable, latest results, due assignments, new resources, announcements. | Must   |
| FR-RPT-04 | Generate attendance, results, and finance reports for a chosen scope and period.                            | Should |
| FR-RPT-05 | Export reports to PDF and CSV.                                                                              | Should |

## 4.18 Public Website (not a PWA)

The public site is a server-rendered Next.js application sharing the backend API. It is intentionally separate from the app and has no offline behaviour.

| ID        | Requirement                                                                                                                                                            | Pri  |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| FR-WEB-01 | A <b>landing page</b> presenting the madrasa.                                                                                                                          | Must |
| FR-WEB-02 | A <b>blog</b> : teachers holding <code>blog.manage</code> author posts in a <b>WYSIWYG editor</b> with formatting styles and <b>multiple font choices</b> ; they get a | Must |

| ID        | Requirement                                                                                                                                                                 | Pri  |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
|           | <b>Manage Blog</b> menu automatically (FR-RBAC-07). Posts support categories/tags and are server-rendered for SEO.                                                          |      |
| FR-WEB-03 | A <b>contact page</b> with a form that stores enquiries and optionally notifies staff.                                                                                      | Must |
| FR-WEB-04 | A public <b>registration page</b> = an admission application; submissions are queued for review.                                                                            | Must |
| FR-WEB-05 | An authorised user <b>reviews and approves</b> an application; approval creates a student record (and a login if portal-enabled), then triggers credential dispatch (4.16). | Must |
| FR-WEB-06 | The marketing site is <b>not</b> a PWA and provides no offline support.                                                                                                     | Must |

## 5. Non-Functional Requirements

---

### 5.1 Performance

- Common interactive screens respond within ~1–2 seconds on a mid-range Android phone over a normal connection.
- Marking attendance for a class is a single batched write; offline capture is instant (no network in the path).
- List screens paginate; large result/finance reports stream or page rather than loading wholesale.

### 5.2 Offline & Reliability

- Attendance capture never depends on connectivity; the outbox syncs idempotently when online (3.4).
- The local store retains 7 days of data and survives app restarts.
- Sync failures are retried with backoff; unsynced items remain visible to the user and, in aggregate, to the Director.

### 5.3 Security & Minors' Privacy

- No plaintext credentials are ever transmitted; onboarding uses 1-day one-time links (3.8).
- Authorization is enforced server-side on every endpoint; least-privilege by default.
- Files are served via time-limited signed URLs; student data never appears on public surfaces.
- Passwords are hashed (argon2/bcrypt); transport is HTTPS; auth endpoints are rate-limited.
- An immutable audit trail covers attendance, marks, finance, permissions, and provisioning.

### 5.4 Usability & Accessibility

- Full **RTL** support for Urdu and LTR for English, switchable per user, with appropriate Nasta'liq and Latin fonts (3.6).
- Designed for **mixed literacy and small screens**: clear icons with labels, large tap targets, minimal typing for routine tasks like attendance.
- Dates display Hijri alongside Gregorian.

### 5.5 Scalability

- Multi-tenant-ready via `madrassa_id` scoping; one madrasa in v1, more without restructuring (3.3).
- Stateless backend processes scale horizontally behind Coolify's proxy; Postgres, Redis, and MinIO scale independently.

### 5.6 Modularity & Extensibility

- A new feature is a self-contained module (routes, services, models, schemas) that registers its permissions and jobs through shared registries — added without modifying unrelated modules.
- Dynamic structures (grading schemes, form definitions, exam types, categories) are data, not code, so most configuration changes require no deployment.

## 5.7 Localization

- All user-facing strings come from translation resources; no hardcoded display text.
- Content language is a single Director-chosen setting; UI language is per user.

## 5.8 Compatibility

- Targets current Chrome, Edge, Safari, and Firefox; the PWA is installable on Android and desktop.
- S3-compatible storage (MinIO) and standard PostgreSQL/Redis keep the stack portable.

## 5.9 Maintainability & Observability

- Versioned migrations (Alembic); typed schemas (Pydantic) at the API boundary.
- Structured logs and health checks per service; background-job status is inspectable.
- OpenAPI is generated from the backend and drives the client's API types.

## 5.10 Backup & Recovery

- Scheduled `pg_dump` of PostgreSQL and backup of the MinIO `minio-data` volume; both volumes (`db-data`, `minio-data`) are persistent.
- Backups are stored off-host; restores are tested periodically.

## 6. Data Model

This section lists the core entities and key relationships at an ERD level. Every tenant-scoped entity carries `madrasa_id`. Dynamic structures are stored relationally where they need integrity and as JSONB where they are genuinely free-form (noted below).

### 6.1 Core Entities

| Entity                      | Purpose / key fields                                                         |
|-----------------------------|------------------------------------------------------------------------------|
| Madrasa                     | Tenant root. One row in v1; every scoped table references it.                |
| User                        | Login identity: username, password hash, role, preferred_language, status.   |
| Permission / UserPermission | Catalogue of capabilities and per-teacher grants (checkboxes).               |
| Program / Class / Section   | Academic hierarchy (e.g., Hifz → Darja → Section).                           |
| Course                      | Subject attached to classes; arbitrary and unlimited.                        |
| AcademicSession             | Named session with Hijri + Gregorian span; one active.                       |
| TeacherProfile              | Linked to a User; employee_code, qualifications, join_date, salary fields.   |
| StudentProfile              | Linked optionally to a User (portal-enabled); admission_number, dob, status. |
| Guardian / StudentGuardian  | Message-only contact (phones, preferred_language) and its links to students. |
| Enrollment                  | Student ↔ program/class/section for a session; drives courses & timetable.   |
| TeacherAssignment           | Teacher ↔ (class + course) for a session; bounds scoped permissions.         |
| StudentAttendance           | Per-day status, marked_at, marked_by, lock state, synced_late flag.          |
| TeacherAttendance           | Per-day status with check_in/check_out, marked_at, lock state.               |
| AttendanceCorrection        | Immutable log: old → new, actor, change time.                                |
| Assignment / Submission     | Class/course assignment, targets, due date; student uploads (object keys).   |
| GradingScheme               | Ordered bands with thresholds/labels (JSONB); selectable per program/exam.   |
| ExamType                    | Per-course assessment type with optional weightage.                          |
| Mark / Result               | Per-student per-assessment scores; rolled-up course and session results.     |
| TimetableSlot               | Class/section day-period-course-teacher entries (manual).                    |
| Holiday / Leave             | Calendar exclusions and approved leave records.                              |
| ResourceCategory / Resource | Categorised notes, files (object keys), and video links; visibility scope.   |
| Form / FormField /          | Form definition (fields incl. type/required, JSONB), opened to               |



| Entity                       | Purpose / key fields                                                     |
|------------------------------|--------------------------------------------------------------------------|
| FormResponse                 | classes, and responses.                                                  |
| Announcement                 | Targeted notice (all/class), schedule/expiry.                            |
| Payment / Donor / Donation   | Voluntary student contributions and external community donations.        |
| SalaryRecord / SalaryPayment | Salary figure and disbursement history (separate from Finance).          |
| MessageTemplate / MessageLog | Bilingual templates with variables; dispatch log.                        |
| FileObject                   | Storage key, content type, owner/scope, for any uploaded/generated file. |
| AuditLog                     | Cross-cutting immutable record of sensitive changes.                     |

## 6.2 Key Relationships

- A **User** is one of Principal / Teacher / Student; a Teacher has a **TeacherProfile**, a portal-enabled Student has a **StudentProfile** plus a User, a non-portal Student has only a StudentProfile.
- **Enrollment** ties a student to program/class/section for a session and thereby to **Courses** and a **Timetable**; **TeacherAssignment** ties a teacher to class+course and bounds their scoped permissions.
- **Attendance** rows reference a student/teacher, a date, and a session; **AttendanceCorrection** rows reference the attendance row they amend.
- **ExamType** and **Mark** reference a course and student; results map through the active **GradingScheme**.
- **Guardian** links many-to-many with **Student** via **StudentGuardian**; messaging reads the guardian's preferred language.
- **Payment** references a student; **Donation** references a **Donor**, never a student.

## 6.3 Notes on Dynamic Structures

- **GradingScheme** bands, **Form** field definitions, and message-template variables are stored as JSONB: free-form, versioned with the row, and editable without migrations.
- Core relationships (enrolment, assignment, attendance, marks, finance) stay relational to preserve integrity and support reporting.

## 7. External Interface Requirements

---

### 7.1 User Interfaces

- **Principal:** configuration, people management, permission checkboxes, dashboards, finance, reports.
- **Teacher:** assigned classes, attendance capture (offline-capable), assignments, marks, resources, forms, and — if granted — supervisory and blog screens.
- **Student (portal):** personal dashboard, timetable, results, assignments, resources, forms, announcements.
- All app screens honour the active language and direction (RTL/LTR).

### 7.2 Software Interfaces

- **PostgreSQL** via SQLAlchemy (async) — primary datastore.
- **MinIO (S3 API)** via the storage abstraction — files; local-disk driver as fallback.
- **Redis** — cache, sessions, and the ARQ job queue.
- **WhatsApp** — v1 via [wa.me](#) click-to-chat links (no server credentials); Phase 2 via the Business Cloud API behind the same messaging interface.

### 7.3 Application Programming Interface

- A **versioned REST API** (e.g., `/api/v1/...`) consumed by the PWA and the marketing site; OpenAPI is generated from the backend.
- Endpoints enforce permissions server-side and are tenant-scoped; sync endpoints accept idempotency keys and past-dated offline records per 3.4.

### 7.4 Communication Interfaces

- HTTPS for all client-server traffic (TLS terminated at Coolify's proxy).
- Outbound links to WhatsApp via the [wa.me](#) scheme; signed URLs for file downloads.

## 8. Roadmap & Phasing

---

### 8.1 v1 Scope

Everything in Section 4 marked Must/Should: identity and granular permissions; configurable academic structure and sessions; teacher and student records; offline-first attendance with the 23:59 lock and correction logs; salary (separate); class/course assignment; assignments and submissions; dynamic assessments, grading schemes, published results, and result-card PDFs; manual timetable with holidays and leave; resources; the simple form builder; announcements; the voluntary finance ledger; [wa.me](#) messaging; dashboards and reports; and the non-PWA marketing site with a teacher-authored blog.

### 8.2 Phase 2

- **WhatsApp Cloud API:** server-sent automation, approved templates, and file attachments.
- **Guardian portal** (read-only) by promoting guardian contacts to logins.
- **Payroll:** absence-linked deductions and full salary processing.
- **Automatic timetable generation** from constraints.
- **Multi-tenant activation:** onboarding additional madaris on the existing scoped schema.
- **Per-period attendance,** push notifications, and richer analytics.

### 8.3 Future / Out of Scope

- Hostel/boarding management, online payment gateways, and a general-purpose CMS are not planned for this product at present.

## Appendix A — Permission Catalogue (starting set)

The Principal toggles these per teacher. **Scope** indicates whether a capability is limited to the teacher's assignments or applies institution-wide. The catalogue is extensible as modules are added.

| Permission key                                                         | Grants                                              | Scope         |
|------------------------------------------------------------------------|-----------------------------------------------------|---------------|
| teachers.add / teachers.edit / teachers.view                           | Manage teacher records                              | Global        |
| teachers.attendance.manage                                             | Mark/correct teacher attendance                     | Global        |
| teachers.salary.manage                                                 | Maintain salary & payments                          | Global        |
| students.add / students.edit / students.view                           | Manage student records                              | Global        |
| students.provision / students.send_credentials                         | Create logins & send credentials                    | Global        |
| students.attendance.manage                                             | Mark/correct student attendance                     | Scoped        |
| academics.manage                                                       | Programs, classes, sections, courses, sessions      | Global        |
| assignments.assign_teacher                                             | Assign teachers to class+course                     | Global        |
| attendance.take                                                        | Take attendance                                     | Scoped        |
| attendance.edit_locked                                                 | Override a locked day (Principal-level)             | Global        |
| assignments.create                                                     | Create/grade assignments                            | Scoped        |
| assignments.view_all / assignments.manage_all / assignments.create_any | Supervise assignments across classes                | Global        |
| assessments.exam_types.manage                                          | Create exam types & weightage                       | Scoped        |
| assessments.marks.enter                                                | Enter marks                                         | Scoped        |
| assessments.results.publish                                            | Publish results                                     | Global        |
| grading.schemes.manage                                                 | Define grading schemes                              | Global        |
| timetable.manage                                                       | Build timetables; holidays & leave                  | Global        |
| resources.manage                                                       | Manage the resource library                         | Global        |
| forms.create / forms.responses.view                                    | Build forms; view responses                         | Scoped/Global |
| announcements.post                                                     | Post announcements (all/class)                      | Global        |
| finance.manage / finance.reports.view                                  | Record income/donations; view finance               | Global        |
| messaging.send                                                         | Send WhatsApp reports/credentials                   | Scoped        |
| blog.manage                                                            | Author/manage blog posts (reveals Manage Blog menu) | Global        |

## Appendix B — Glossary

| Term                                              | Meaning                                                                             |
|---------------------------------------------------|-------------------------------------------------------------------------------------|
| Madrasa                                           | An Islamic educational institution; the single tenant in v1.                        |
| Hifz                                              | Programme of Qur'an memorisation.                                                   |
| Nazira                                            | Programme of Qur'an reading/recitation.                                             |
| Dars-e-Nizami                                     | The classical multi-year Islamic studies curriculum.                                |
| Darja                                             | A class/year level within a programme.                                              |
| Mumtaz / Jayyid Jiddan / Jayyid / Maqbool / Rasib | Example madrasa grade bands: excellent / very good / good / pass / fail.            |
| Hijri                                             | The Islamic lunar calendar; shown alongside Gregorian dates.                        |
| Kafāla                                            | Sponsorship — here, a community member supporting a student via voluntary donation. |
| PWA                                               | Progressive Web App — installable, offline-capable web application.                 |
| RTL / LTR                                         | Right-to-left (Urdu) / left-to-right (English) layout direction.                    |
| i18n                                              | Internationalisation — supporting multiple languages and locales.                   |
| RBAC                                              | Role-Based Access Control; here, granular per-teacher permissions.                  |
| Modular monolith                                  | One deployable application split into clean, independent internal modules.          |
| JSONB                                             | PostgreSQL binary JSON type used for free-form configurable data.                   |
| Outbox / idempotency key                          | Local queue of offline writes / a key making re-sends safe.                         |
| wa.me / Cloud API                                 | WhatsApp click-to-chat links (v1) / WhatsApp Business Cloud API (Phase 2).          |
| Signed URL                                        | A time-limited link granting temporary access to a stored file.                     |

## Appendix C — Sample WhatsApp Templates

---

Templates carry variable placeholders and exist in both languages; the active one is chosen by the recipient's preferred language. (Urdu shown for illustration; exact wording to be finalised with the madrasa.)

### *Performance report — English*

```
Assalamu Alaikum {guardian_name},  
Result for {student_name} ({class_name}), {session}:  
{summary_line}  
Full result card: {result_link}  
– {madrasa_name}
```

### *Performance report — Urdu*

```
السلام علیکم {guardian_name},  
{student_name} ({class_name}) کا نتیجہ {session}:  
{summary_line}  
مکمل نتیجہ کارڈ: {result_link}  
– {madrasa_name}
```

### *Credentials dispatch — English*

```
Assalamu Alaikum,  
Portal access for {student_name}.  
Username: {username}  
Set your password (valid 24h): {setup_link}  
– {madrasa_name}
```

## Appendix D — Grading Scheme Examples

| Scheme        | Bands (high → low)                                                                   |
|---------------|--------------------------------------------------------------------------------------|
| Madrasa bands | Mumtaz $\geq 90$ · Jayyid Jiddan 80–89 · Jayyid 65–79 · Maqbool 50–64 · Rasib $< 50$ |
| Letter grades | A+ $\geq 90$ · A 80–89 · B 70–79 · C 60–69 · D 50–59 · F $< 50$                      |
| Percentage    | Raw percentage with a configurable pass mark (e.g., 50%)                             |

Thresholds and labels are configured by the Director and stored as data; the table shows illustrative values only.

## Appendix E — Deployment Files

---

The Coolify deployment files in Section 3.9 are also provided as standalone artifacts for direct use in a repository: `docker-compose.yml`, `backend/Dockerfile`, `app/Dockerfile`, `web/Dockerfile`, and `.env.example`. Secrets are set in Coolify rather than committed; the `.env.example` lists variable names only.